Coms 472/572 Recitation

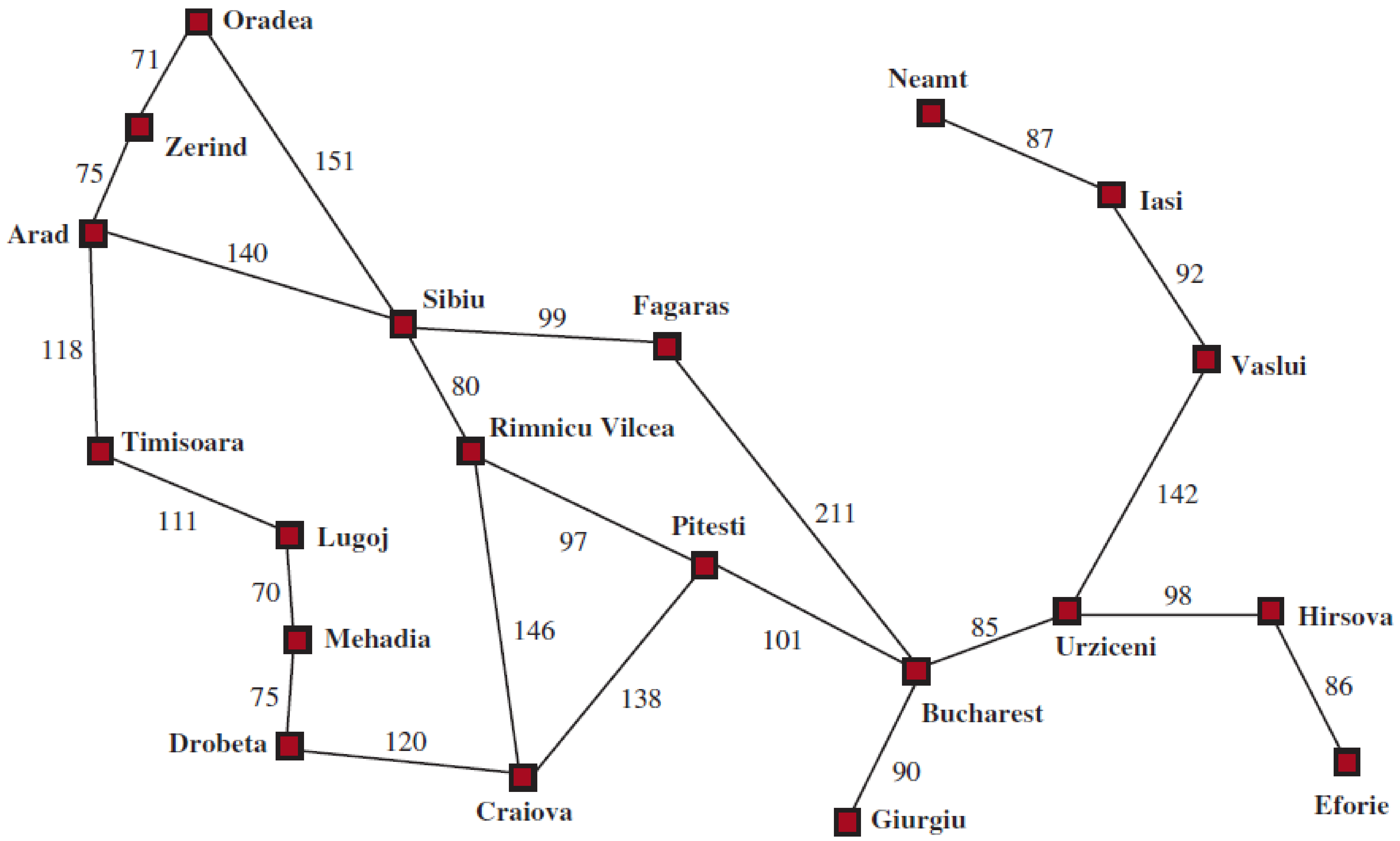
EXAM REVIEW(WEEK-6)

Xuye Wu, Computer Science department

Example

Suppose an agent starts in Arad and wants to reach Bucharest.

Formulate this as a search problem



Example

1. State Space
Each state represents the agent being in a particular city.

$$State = In(city)$$

2. Initial State

$$Initial\ State = In(Arad)$$

3. Actions
From any city, the agent can travel to any directly connected neighboring city.

$$ACTIONS(In(x)) = \{Go(y) \mid y\ is\ a\ neighbor\ of\ x\}.$$

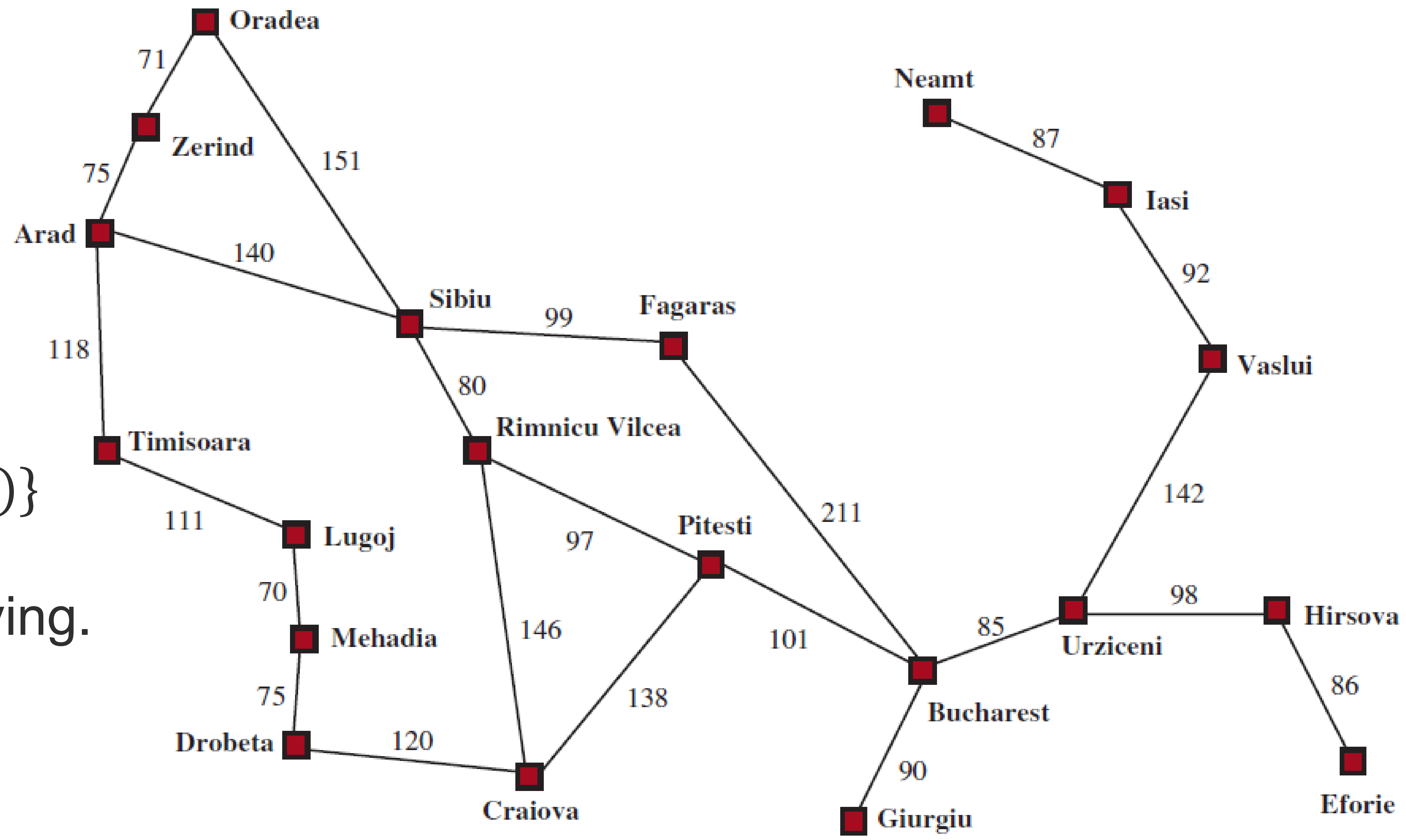
Example:

$$ACTIONS(In(Arad)) = \{Go(Zerind), Go(Sibiu), Go(Timisoara)\}$$

4. Transition Model
The transition function $RESULT(s, a)$ returns the new city after moving.

$$RESULT(In(x), Go(y)) = In(y)$$

Example: $RESULT(In(Oradea), Go(Zerind)) = In(Zerind)$
5. Goal Test
Goal: $In(Bucharest)$
6. Cost Function
The path cost is the sum of road distances along the chosen route.



Example

The memory requirement for the iterative deepening search is linear in the depth d at which the goal state is found.

Example

The memory requirement for the iterative deepening search is linear in the depth d at which the goal state is found.

True

In Iterative Deepening Search (IDS), we repeatedly run Depth-Limited DFS with increasing depth limits: 0,1,2,…,d

The space complexity is: $O(bd)$

b= branching factor

d= depth of goal

b is constant, so iterative deepening search is linear in the depth d

Example

A best-first search terminates as soon as a goal state is found.

Example

A best-first search terminates as soon as a goal state is found.

True

- Best-first search checks for the goal when a node is selected for expansion (i.e., when it is removed from OPEN).

- Therefore, as soon as the goal becomes the best node in the frontier, it is expanded and the algorithm stops.

Example

The A* algorithm using an inadmissible heuristic can never find an optimal solution path.

Example

The A* algorithm using an inadmissible heuristic can never find an optimal solution path.

False

An inadmissible heuristic means: for at least one node — i.e., it overestimates the true cost to the goal.

Even if a heuristic overestimates somewhere:

- The overestimation might not affect the nodes on the optimal path.
- Or the optimal path might still end up with the smallest $f(n)=g(n)+h(n)$.

In those cases, A* will still expand the optimal path first and return the correct solution.

Example

	Uniform-cost	Iterative deepening	A* using a zero g-cost function
Complete?			
Optimal cost?			

Example

Uniform-Cost Search

Complete: Yes — if step costs are positive, it will eventually expand nodes in increasing path cost order and reach the goal.

Optimal: Yes — with equal step costs, the first goal found (at minimum depth) is also the minimum-cost solution

Iterative Deepening

Complete: Yes — it increases depth limit until the goal depth is reached.

Optimal: Yes — with unit step costs, it finds the shallowest goal first.

Example

A* with zero g-cost (i.e., $f(n)=h(n)$)

This is **Greedy Best-First Search**.

Complete: No — can get stuck following misleading heuristic paths.

Optimal: No — does not consider path cost $g(n)$, so may return suboptimal solution.

Example

	Uniform-cost	Iterative Deepening	A* using a zero g-cost function
Complete?	Yes	Yes	No
Optimal cost?	Yes	Yes	No

Example : heuristic

Consider a search employing some heuristic function h. Let n and n' be two nodes in the search tree such that $n'$is generated from n by applying some action a. Suppose that $h(n) = 12$ and $h(n') = 8$. The cost of a is 3, denoted $c(n, a, n') = 3$.
Is the used heuristic consistent?

Example : heuristic

Consider a search employing some heuristic function h. Let n and n' be two nodes in the search tree such that $n'$is generated from n by applying some action a. Suppose that $h(n)$=12 and $h(n')$=8. The cost of a is 3, denoted $c(n, a, n') = 3$.
Is the used heuristic consistent?

No, the heuristic is not consistent because $12 > 3 + 8$, violating the consistency condition $h(n) \leq c(n, a, n') + h(n')$.

Example :

Both breadth-first search (BFS) and depth-first search (DFS) are conducted in a state space. Let b be the branching factor, d be the depth of the (unique) goal state, and m be the maximum depth of the resulting search tree.

In big-O notation, what are the runtime (time complexity) and space complexity of BFS and DFS?

Example :

Both breadth-first search (BFS) and depth-first search (DFS) are conducted in a state space. Let b be the branching factor, d be the depth of the (unique) goal state, and m be the maximum depth of the resulting search tree.

In big-O notation, what are the runtime (time complexity) and space complexity of BFS and DFS?

$O(b^d)$ in both time and memory for BFS.

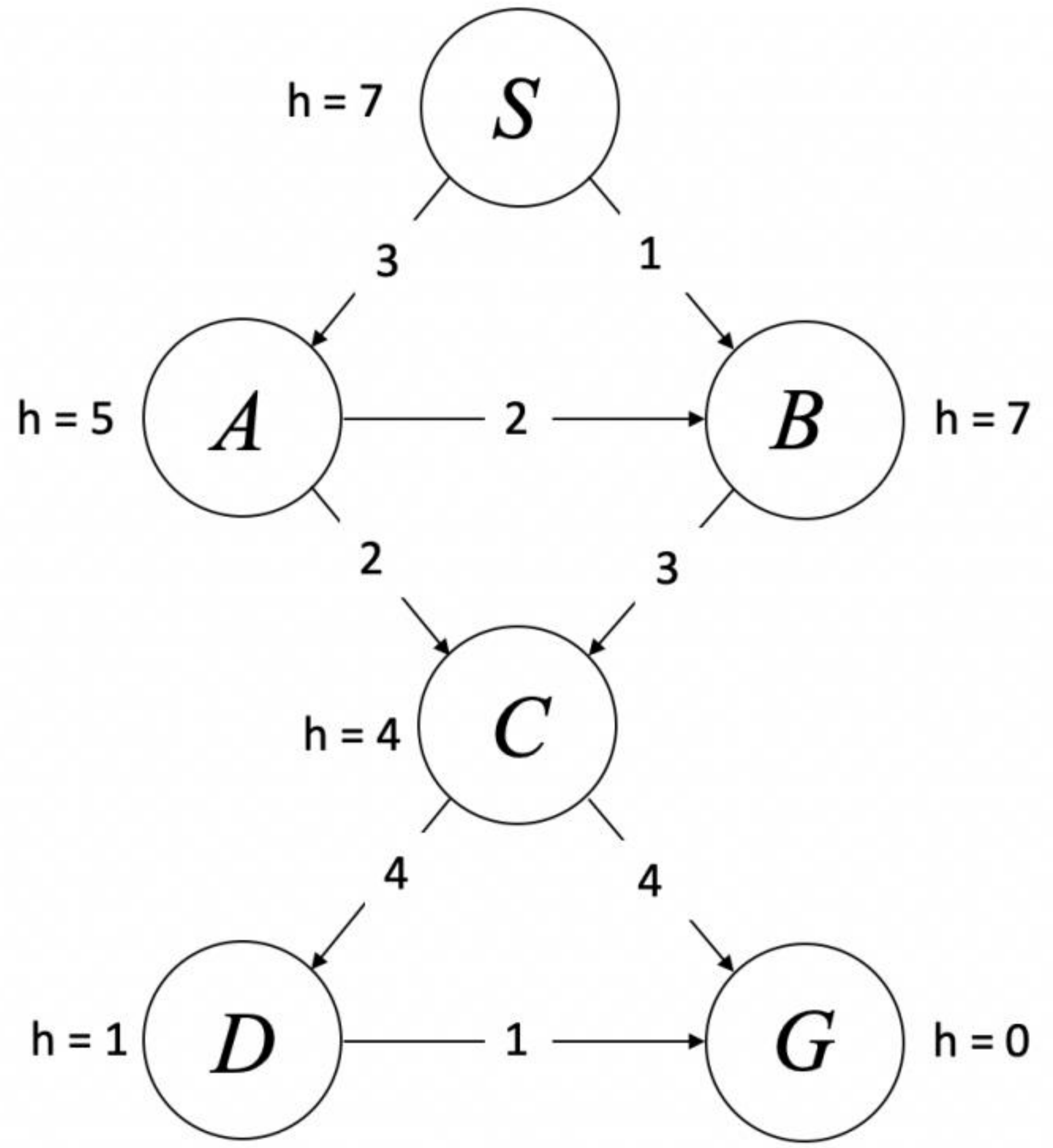
$O(b^m)$ in time and $O(bm)$ in memory for DFS.

Example :

Given the directed, weighted graph on the right (edge labels are step costs) and heuristic values $h(n)$ shown at each node. Start is S and goal is G. Assume ties are broken in alphabetical order.

Run Greedy Best-First and report the solution path and its total path cost.

Run A* and report the solution path and its total path cost.



Example :

Greedy:

S→A→C→G

3+2+4=9

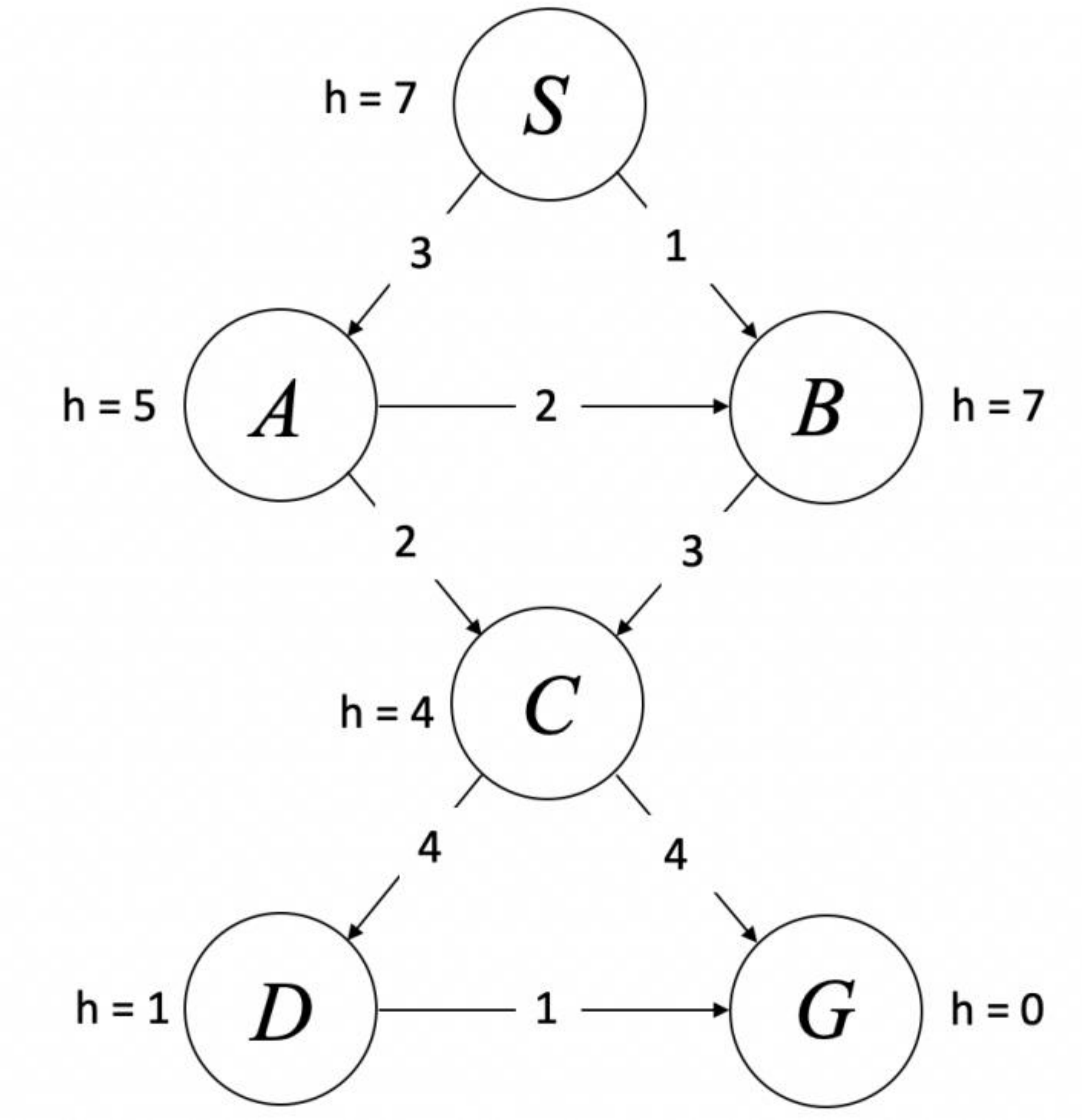
A*:

S→B→C→G

1+3+4=8

Greedy Best-First Search is not guaranteed to find the optimal solution because it relies solely on the heuristic estimate and ignores the path cost

A* combines both the path cost and the heuristic and therefore guarantees optimality when the heuristic is admissible.




That's all Folks!